



Documentação do Projeto — Alexandria: Biblioteca Digital

Sumário

1. [Identificação do Projeto](#)
 2. [Introdução](#)
 3. [Objetivos](#)
 4. [Público-Alvo](#)
 5. [Requisitos Funcionais](#)
 6. [Requisitos Não Funcionais](#)
 7. [Arquitetura do Sistema](#)
 8. [Backend — Alexandria API](#)
 9. [Frontend — Alexandria Web](#)
 10. [Descrição das Telas](#)
 - 10.1 [Telas Desktop](#)
 - 10.2 [Telas Mobile](#)
 11. [Design e Interface](#)
 12. [Autenticação e Segurança](#)
 13. [Integrações Externas](#)
 14. [Infraestrutura e Deploy](#)
 15. [Estrutura do Projeto](#)
 16. [Como Executar o Projeto](#)
 17. [Testes](#)
 18. [Plano de Evolução](#)
 19. [Considerações Finais](#)
-

1. Identificação do Projeto

Campo	Informação
Nome do Projeto	Alexandria
Subtítulo	Biblioteca Digital
Tipo	Aplicação Web Full-Stack

Campo	Informação
Categoria	Gerenciamento de Acervo Literário Pessoal
Frontend	Next.js 16 + TypeScript + Tailwind CSS v4
Backend	Java 17 + Spring Boot 3.4.4
Banco de Dados	MySQL 8
Plataformas	Desktop Web, Mobile Web, PWA
Repositório	Monorepo com alexandria-frontend/ e alexandria-backend/

2. Introdução

O **Alexandria** é uma plataforma web full-stack desenvolvida para suprir uma necessidade comum entre leitores ávidos: a falta de um sistema centralizado, elegante e funcional para gerenciar acervos literários pessoais.

Inspirado nas grandes bibliotecas históricas — em especial na lendária **Biblioteca de Alexandria**, símbolo de conhecimento e curadoria intelectual — o sistema oferece ao usuário uma experiência editorial sofisticada para catalogar, explorar e acompanhar seus livros.

O projeto nasce da observação de que muitos leitores recorrem a planilhas ou aplicativos genéricos para organizar seus livros, sem acesso a recursos como curadoria temática, leitor de eBooks integrado com progresso automático, login social via Google ou dashboard de estatísticas de leitura. O Alexandria resolve esse problema ao reunir essas funcionalidades em uma interface visualmente refinada, responsiva e alimentada por uma API REST robusta.

O acervo da plataforma é alimentado pelo **Projeto Gutenberg** (via Gutendex API), que disponibiliza milhares de obras em domínio público para leitura gratuita diretamente no navegador.

3. Objetivos

3.1 Objetivo Geral

Desenvolver uma aplicação web full-stack responsiva que permita ao usuário gerenciar seu acervo literário pessoal de forma centralizada, com suporte a leitura de eBooks, curadoria, controle de progresso e autenticação social.

3.2 Objetivos Específicos

- **Cadastro e autenticação** de usuários com suporte a login tradicional (JWT) e Google OAuth
 - **Importação de livros** do Projeto Gutenberg via Gutendex API, com sincronização automática agendada
 - **Busca e descoberta** de novas obras por título ou autor
 - **Acervo pessoal** com adição, remoção e filtragem por status (Para Ler, Lendo, Concluído)
 - **Leitor de eBooks** integrado (formato EPUB) com progresso sincronizado automaticamente com o backend
 - **Cache offline de livros** via IndexedDB com política LRU para leitura sem conexão
 - **Dashboard** com estatísticas reais de leitura (livros lidos, em progresso, tempo estimado)
 - **Perfil do usuário** com edição de dados e alteração de senha
 - **Aplicação web progressiva (PWA)** com Service Worker e manifest para instalação no dispositivo
 - **Experiência responsiva** adequada em dispositivos desktop e mobile
 - **Arquitetura limpa** (Clean Architecture / Ports & Adapters) no backend para manutenibilidade e testabilidade
-

4. Público-Alvo

O Alexandria é voltado para:

- **Leitores frequentes** que possuem um acervo considerável e desejam organizá-lo digitalmente
- **Estudantes e pesquisadores** que precisam controlar leituras acadêmicas e referências bibliográficas
- **Colecionadores de livros** interessados em catalogar edições e raridades

Perfil demográfico: usuários com habilidade básica em navegação web, de 16 a 60 anos, com interesse em literatura, tecnologia e organização pessoal.

5. Requisitos Funcionais

ID	Requisito	Status	Prioridade
RF01	O sistema deve permitir cadastro e login de usuários (tradicional + Google OAuth)	✓ Implementado	Alta
RF02	O sistema deve permitir busca de livros por título ou autor	✓ Implementado	Alta
RF03	O sistema deve exibir o acervo pessoal do usuário com filtros por status	✓ Implementado	Alta
RF04	O sistema deve indicar o progresso de leitura de cada livro com barra visual	✓ Implementado	Média
RF05	O sistema deve exibir um dashboard com estatísticas de leitura e tempo estimado	✓ Implementado	Média
RF06	O sistema deve oferecer um leitor de EPUB integrado com progresso automático	✓ Implementado	Média
RF07	O sistema deve importar livros da Gutendex API	✓ Implementado	Alta

ID	Requisito	Status	Prioridade
RF08	O sistema deve sincronizar automaticamente o acervo da Gutendex via job agendado	✓ Implementado	Média
RF09	O sistema deve permitir edição de perfil (nome, username) e alteração de senha	✓ Implementado	Média
RF10	O sistema deve funcionar como PWA (instalável, cache offline de EPUBs)	✓ Implementado	Baixa
RF11	O sistema deve ter controle de acesso por papéis (ADMIN pode gerenciar acervo)	✓ Implementado	Média
RF12	O sistema deve funcionar adequadamente em dispositivos móveis	✓ Implementado	Alta

6. Requisitos Não Funcionais

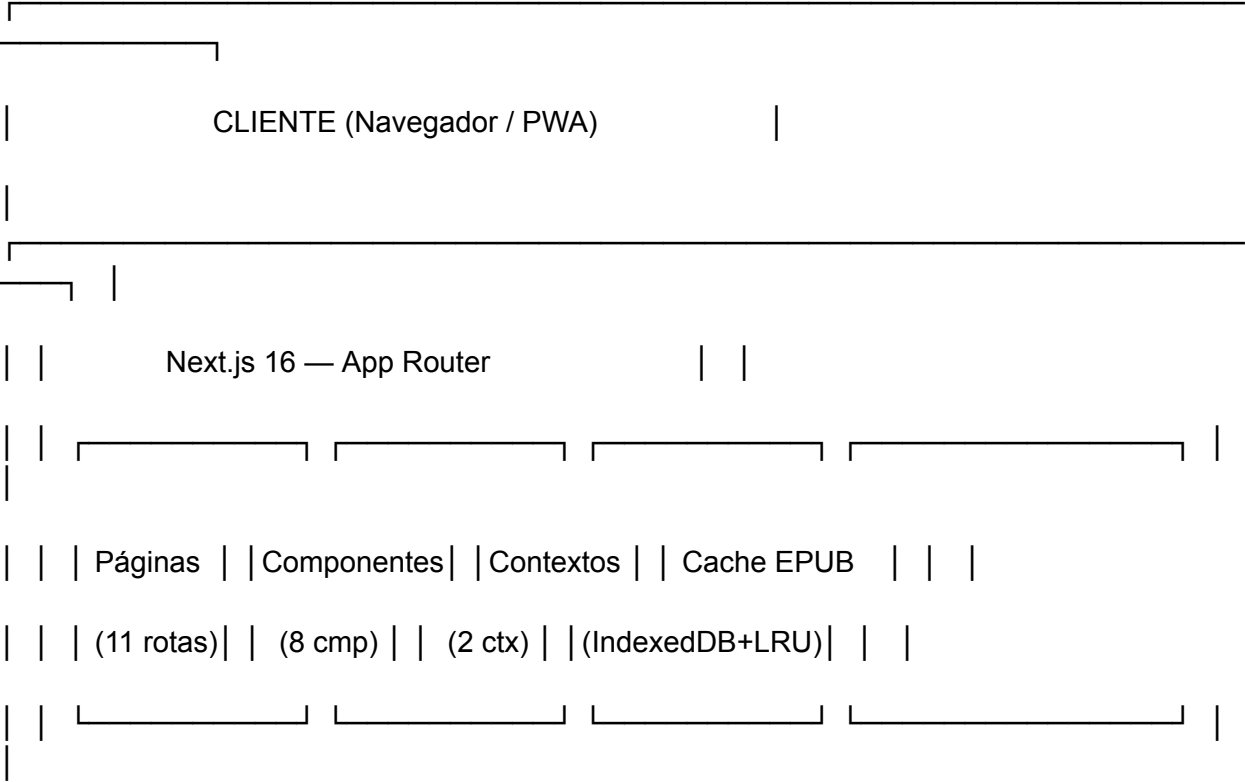
ID	Requisito	Categoria	Status
RNF01	A API REST deve responder em menos de 500ms (p95)	Desempenho	✓
RNF02	O sistema deve ser responsivo para telas	Usabilidade	✓

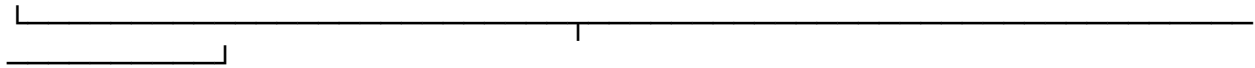
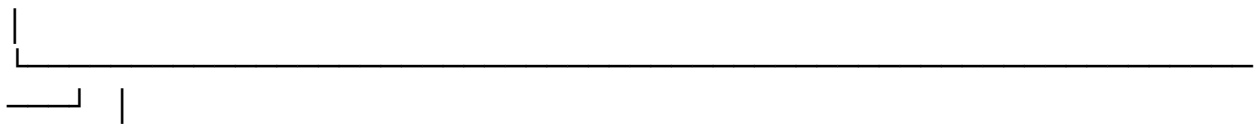
ID	Requisito	Categoria	Status
	a partir de 390px de largura		
RNF03	O código frontend deve ser escrito em TypeScript com tipagem estrita	Manutenibilidade	✓
RNF04	O backend deve seguir Clean Architecture (Ports & Adapters)	Manutenibilidade	✓
RNF05	A paleta de cores deve seguir os tokens de design definidos no Figma	Consistência	✓
RNF06	O sistema deve utilizar fontes do Google Fonts com carregamento otimizado	Desempenho	✓
RNF07	A navegação deve funcionar sem recarregamento de página (SPA)	Usabilidade	✓
RNF08	O cache de EPUBs deve suportar no mínimo 30 livros (~250MB) em IndexedDB	Desempenho	✓
RNF09	A senha do usuário deve ser armazenada com hash BCrypt	Segurança	✓
RNF10	O token JWT deve expirar em 24 horas	Segurança	✓

ID	Requisito	Categoria	Status
RNF11	O schema do banco deve ser versionado (Flyway) para produção	Manutenibilidade	 Pós-MVP
RNF12	O backend deve ter cobertura de testes mínima de 80% (JaCoCo)	Qualidade	
RNF13	A API deve ser documentada com Swagger/OpenAPI	Documentação	

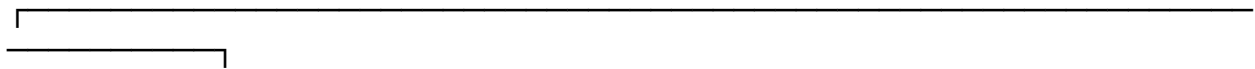
7. Arquitetura do Sistema

7.1 Visão Geral





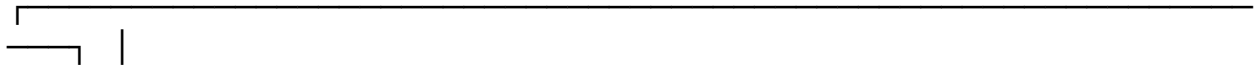
| HTTP REST (JSON) + JWT Bearer Token



| ALEXANDRIA API (Spring Boot 3.4.4) |

| |

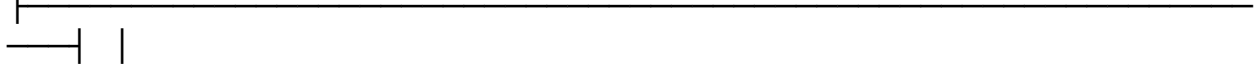
|



| | adapter.in (Controllers) — Auth, Book, UserBooks, Job, | |

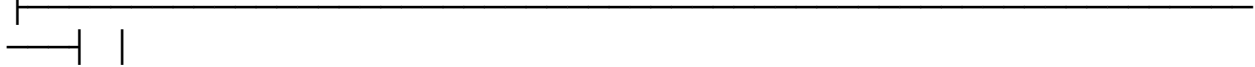
| | Profile | |

|



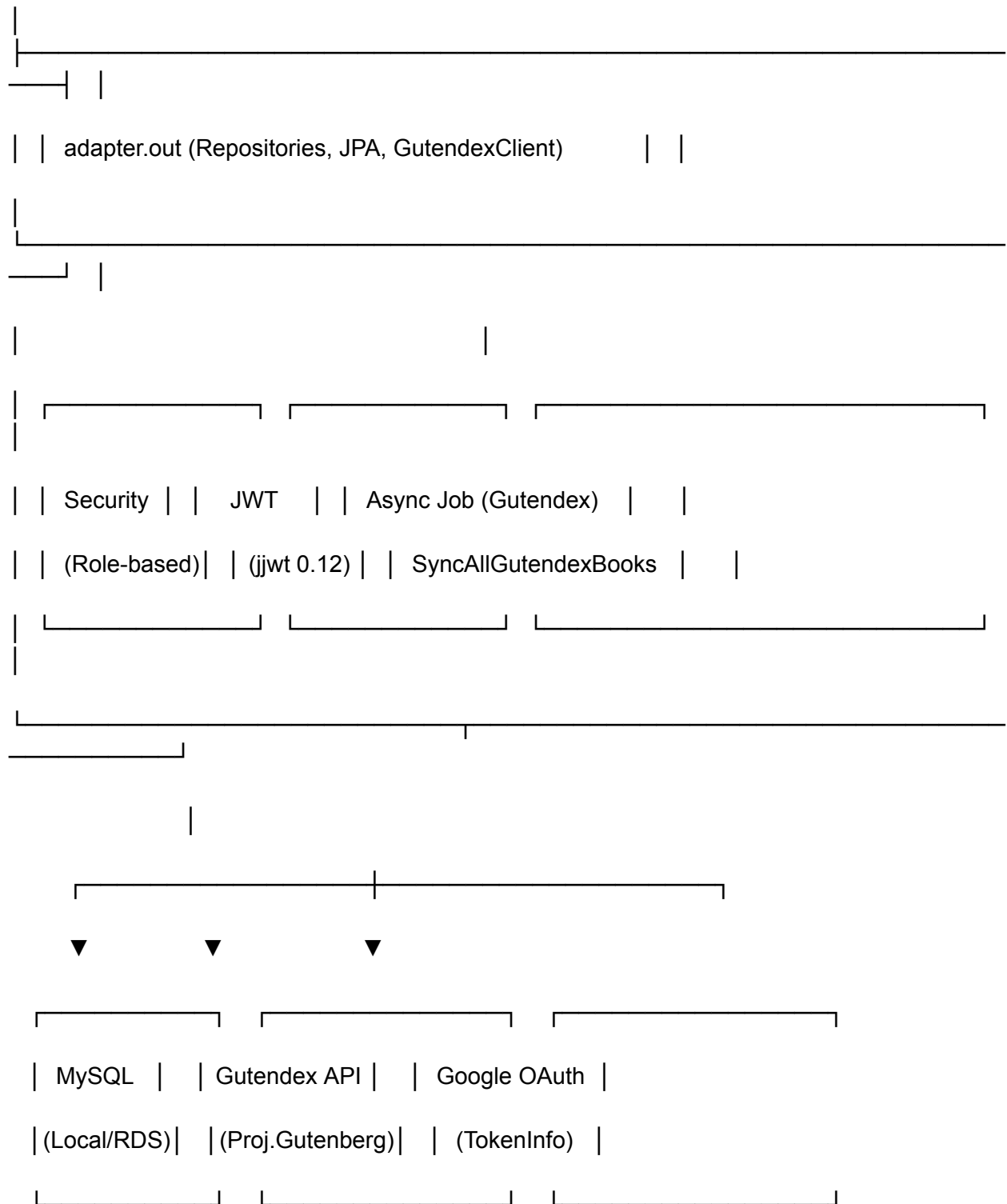
| | application (Use Cases) — 18 casos de uso | |

|



| | domain (Entities, VOs, Ports) — Book, Author, User, | |

| | UserBooks, AuthenticatedUser | |



7.2 Tecnologias

Frontend

Tecnologia	Versão	Propósito
Next.js	16 (canary)	Framework React com App Router, SSR/SSG
TypeScript	5.x	Tipagem estática
Tailwind CSS	4.x	Estilização utilitária com tokens de design
React	19 (canary)	Biblioteca de UI
Lucide React	—	Ícones SVG (tree-shaking)
@react-oauth/google	—	Google OAuth login button
react-reader	—	Leitor de EPUB no navegador
Google Fonts (next/font)	—	Fontes Manrope, Playfair Display, Noto Serif

Backend

Tecnologia	Versão	Propósito
Java	17	Linguagem
Spring Boot	3.4.4	Framework principal
Maven	—	Gerenciamento de dependências
MySQL 8	—	Banco de dados relacional
Spring Data JPA + Hibernate	—	ORM e acesso a dados
Flyway	—	Migrações de banco (pós-MVP)

Tecnologia	Versão	Propósito
Spring Security	—	Autenticação JWT + autorização role-based
JWT (jjwt)	0.12.6	Geração/validação de tokens
SpringDoc OpenAPI	2.7.0	Documentação Swagger
Testcontainers	1.19.0	Testes de integração com MySQL
JaCoCo	0.8.12	Cobertura de testes
AWS Parameter Store	2.4.4	Configuração externalizada (AWS)

Infraestrutura

Tecnologia	Propósito
Docker + Docker Compose	Containerização local
AWS ECS (Fargate)	Orquestração de containers em produção
AWS RDS (MySQL)	Banco de dados gerenciado
AWS EventBridge Scheduler	Job de sincronização Gutendex
AWS Parameter Store	Configuração externalizada
Kubernetes (K8s)	Orquestração alternativa

7.3 Padrões Arquiteturais

Backend — Clean Architecture (Ports & Adapters)

O backend segue estritamente o padrão **Hexagonal Architecture**, onde as dependências apontam para dentro — o Domínio nunca depende de Infraestrutura:

Controllers (adapters de entrada)

→ Use Cases (aplicação pura, sem anotações Spring)

→ Ports (interfaces no domínio)

→ RepositoryImpl / ApiClientImpl (adapters de saída)

Regras da camada de aplicação:

- Use Cases são classes POJO registradas como `@Bean` em `BeanConfiguration.java`
- Cada Use Case tem um único método público `execute()`
- Use Cases retornam DTOs de saída (nunca objetos de domínio para o adapter)
- Use Cases nunca dependem de implementações concretas — sempre das interfaces do domínio

Frontend — Next.js App Router

- **Route Groups** (`((main)/, (auth)/)`) — agrupa rotas que compartilham layout sem afetar a URL
 - **Server/Client split** — páginas são Server Components por padrão; apenas componentes com estado ou eventos de browser recebem `"use client"`
 - **PWA** — Service Worker registrado para cache e instalação
 - **Cache IndexedDB** — sistema LRU para EPUBs com até 30 itens e 250MB
-

8. Backend — Alexandria API

8.1 Estrutura de Pacotes

com.pucsp.alexandria

— AlexandriaApplication.java	
— domain/	# Núcleo do domínio
— book/	# Agregado Book (Book, BookId, BookSource)
— author/	# Agregado Author (Author, AuthorId)
— user/	# Agregado User (com Role: USER ADMIN)
— userbook/	# Agregado UserBooks (TOREAD READING DONE)
— shared/	# Id<T>, Email, AuthenticatedUser, Port, DomainException
— application/	# Casos de uso

- | └─ book/ # CRUD + Search + SyncAllGutendex
- | └─ auth/ # Register, Authenticate, GoogleAuth
- | └─ profile/ # GetProfile, UpdateProfile, UpdatePassword
- | └─ userbooks/ # Add, List, Update, Remove, GetByBookId
- | └─ adapter/
- | └─ in/
- | | └─ rest/ # Controllers (5) + DTOs
- | | └─ job/ # SyncGutendexJobService (@Async)
- | └─ out/persistence/ # Impls, JPA Entities, Mappers, GutendexClient
- | └─ config/ # Security, JWT, CORS, Async, OpenAPI
- | └─ advice/ # GlobalExceptionHandler

8.2 Controllers REST

Endpoints Públicos

Método	Path	Descrição
POST	/auth/register	Registrar novo usuário
POST	/auth/login	Login (retorna token JWT)
POST	/auth/google	Login com Google OAuth
GET	/books/search?query=	Buscar livros por título/autor
GET	/books	Listar livros (paginado, ordenável)
GET	/books/{id}	Buscar livro por ID
GET	/actuator/health	Health check

Método	Path	Descrição
GET	/swagger-ui/**, /api-docs/**	Documentação Swagger

Endpoints ROLE_ADMIN

Método	Path	Descrição
POST	/books	Importar página da Gutendex
PUT	/books/{id}	Atualizar dados de um livro
DELETE	/books/{id}	Deletar livro
POST	/api/jobs/sync-gutendex	Disparar sincronização completa

Endpoints Autenticados (qualquer role)

Método	Path	Descrição
GET	/user-books	Listar livros da coleção (filtro por status)
GET	/user-books/book/{bookId}	Buscar relação user-book específica
POST	/user-books	Adicionar livro à coleção
PUT	/user-books/{id}	Atualizar status/progresso/rating
DELETE	/user-books/{id}	Remover livro da coleção
GET	/profile/me	Obter dados do perfil
PUT	/profile/me	Atualizar username/firstName/lastName
PUT	/profile/password	Alterar senha

8.3 Modelo de Dados

-- Tabela principal de livros

books

|— id BIGINT PK AUTO_INCREMENT

|— title VARCHAR(500) NOT NULL

|— gutendex_id BIGINT UNIQUE NULL ← ID externo da Gutendex

|— download_url LONGTEXT NULL ← URL de download (ePub)

|— cover_url LONGTEXT NULL ← URL da capa

|— languages LONGTEXT NULL ← Ex: "pt,en"

|— subjects LONGTEXT NULL ← Ex: "Romance;Ficção"

|— download_count INT NULL ← Contagem de downloads

|— publisher_id BIGINT NULL ← FK para editora (não implementado)

|— source VARCHAR(255) NOT NULL ← "GUTENDEX" ou "LOCAL"

-- Autores

authors

|— id BIGINT PK AUTO_INCREMENT

|— name VARCHAR(255) NOT NULL

|— birth_year INT NULL

|— death_year INT NULL

-- Relacionamento N:N livros ↔ autores

book_authors

|— book_id BIGINT NOT NULL FK → books.id

|— author_id BIGINT NOT NULL FK → authors.id

|— PRIMARY KEY (book_id, author_id)

-- Usuários

users

|— id BIGINT PK AUTO_INCREMENT

|— username VARCHAR(255) UNIQUE NOT NULL

|— first_name VARCHAR(255) NOT NULL

|— last_name VARCHAR(255) NOT NULL

|— email VARCHAR(255) UNIQUE NOT NULL

|— password VARCHAR(255) NOT NULL ← BCrypt hash

|— role VARCHAR(20) DEFAULT 'USER' ← USER | ADMIN

|— created_at DATETIME

-- Coleção de livros do usuário

user_books

|— id BIGINT PK AUTO_INCREMENT

|— user_id BIGINT NOT NULL FK → users.id

|— book_id BIGINT NOT NULL FK → books.id

|— status VARCHAR(20) DEFAULT 'toread' ← toread|reading|done

|— progress INT NULL ← 0–100 (para READING)

|— rating INT NULL ← 0–5 (para DONE)

|— created_at DATETIME

|— UNIQUE (user_id, book_id)

8.4 Regras de Validação (UserBooks)

Status	progress	rating
TOREAD	Deve ser <code>null</code>	Deve ser <code>null</code>
READING	Obrigatório (0–100)	Deve ser <code>null</code>
DONE	Deve ser <code>null</code>	Obrigatório (0–5)

8.5 Job de Sincronização Gutendex

O sistema possui um job assíncrono (`SyncAllGutendexBooksUseCase`) que:

1. Itera por todas as páginas da API Gutendex (`https://gutendex.com/books?page={n}`)
2. Para cada livro, verifica duplicidade por `gutendexId`
3. Cria autores na tabela `authors` se não existirem (por nome)
4. Cria o livro e associa os autores via `book_authors`
5. Pode ser disparado manualmente via `POST /api/jobs/sync-gutendex` ou agendado pelo AWS EventBridge

9. Frontend — Alexandria Web

9.1 Páginas

Rota	Descrição	Fonte dos Dados
<code>/</code>	Redireciona para <code>/explorar</code>	—
<code>/login</code>	Página de login tradicional	API <code>/auth/login</code>
<code>/registrar</code>	Página de cadastro	API <code>/auth/register</code>
<code>/explorar</code>	Busca e descoberta de livros	API <code>/books</code> , <code>/books/search</code>
<code>/explorar/{id}</code>	Detalhes do livro + adicionar à biblioteca	API <code>/books/{id}</code>

Rota	Descrição	Fonte dos Dados
/biblioteca	Acervo pessoal com filtros e remoção	API /user-books
/leitor	Leitor estático (placeholder)	Dados mockados
/leitor/[id]	Leitor EPUB real com progresso	API + useEpub hook
/dashboard	Estatísticas de leitura	API /user-books
/configuracoes	Edição de perfil e senha	API /profile/me, /profile/password
/suporte	FAQ e contato	Conteúdo estático

9.2 Componentes

Componente	Função
Sidebar	Navegação lateral (desktop)
Header / MobileHeader	Barra superior com logo e busca
BottomNav	Navegação inferior (mobile)
BookCard	Card reutilizável de livro
LoginModal	Modal de login com Google OAuth
AuthModalTrigger	Abre modal via parâmetro ?auth=1
GoogleProvider	Provider do Google OAuth
ServiceWorkerRegister	Registro do Service Worker (PWA)

9.3 Contextos

Contexto	Função
AuthContext	Estado global de autenticação (login, logout, loginWithGoogle, updateUsername)
AuthModalContext	Controle de abertura/fechamento do modal de login

9.4 Hooks e Utilitários

Módulo	Função
<code>lib/api.ts</code>	Cliente HTTP completo para todas as rotas da API
<code>lib/epub-cache.ts</code>	Cache de EPUBs em IndexedDB com política LRU (30 itens, 250MB)
<code>hooks/useEpub.ts</code>	Hook que gerencia download, cache e heartbeat de EPUBs
<code>lib/proxy.ts</code>	Configuração de proxy para ambiente dev
<code>app/api/epub/route.ts</code>	Proxy Next.js para download de EPUBs da Gutendex

9.5 PWA

O Alexandria é uma **Progressive Web Application**:

- **Service Worker** (`public/sw.js`) — cache de assets e suporte offline parcial
 - **Manifest** (`public/manifest.json`) — permite instalação como aplicativo no dispositivo
 - **Ícones** — `icon-192x192.png` e `icon-512x512.png` para splash screen e home screen
-

10. Descrição das Telas

10.1 Telas Desktop

As telas desktop são exibidas em dispositivos com largura de tela igual ou superior a 768px. O layout é composto por uma **sidebar lateral fixa** (256px) de navegação e uma **área de conteúdo principal** com header superior.

10.1.1 Explorar e Adicionar (`/explorar`)

Objetivo: Tela inicial da aplicação e principal ponto de descoberta de novos livros.

Componentes principais:

- **Hero editorial** com título de impacto em três linhas e texto descritivo lateral
- **Campo de busca** com input de texto e botão de ação (consulta `GET /books/search`)
- **Seção de curadorias em destaque** no estilo Bento Grid (4 coleções mockadas)
- **Grade de recomendações** — top 4 livros por download count (`GET /books?sort=downloadCount, desc`)
- Cada card de livro exibe: capa, título, autor, e link para detalhes

Integração com API: Real — busca e listagem via Gutendex API (proxy pelo backend).

10.1.2 Detalhes do Livro (`/explorar/{id}`)

Objetivo: Exibir informações completas de um livro e permitir adicioná-lo à biblioteca.

Componentes principais:

- Capa do livro (ou placeholder "Sem capa")
- Título, autor(es), assuntos (tags), idioma, fonte, contagem de downloads
- Botão "**Ler agora**" — redireciona para `/leitor/{id}` (se `downloadUrl` disponível)
- Botão "**Adicionar à Biblioteca**" — chama `POST /user-books` (se não autenticado, abre modal de login)

Integração com API: Real — `GET /books/{id}` e `POST /user-books`.

10.1.3 Minha Biblioteca (`/biblioteca`)

Objetivo: Visualização e gerenciamento do acervo pessoal do usuário, com dados reais da API.

Componentes principais:

- Título da página com contador total de livros (`userBooks.length`)
- **Campo de busca** para filtrar por título ou autor localmente
- **Filtros horizontais** por status: Todos, Lendo, Concluído, Para Ler
- **Grade de livros** em 4 colunas, cada card exibindo:
 - Capa do livro (com overlay "Ler" se tiver `downloadUrl`)
 - Badge de status: percentual (se `READING`), "Concluído" (se `DONE`)
 - Barra de progresso (se `READING`)
 - Botão de remoção (X no canto superior direito, com confirmação)

- Clique na capa com `downloadUrl` → redireciona para o leitor

Integração com API: Real — `GET /user-books`, `DELETE /user-books/{id}`.

10.1.4 Dashboard (`/dashboard`)

Objetivo: Painel de controle com visão geral da atividade de leitura do usuário, com dados reais da API.

Componentes principais:

- **Cards de estatísticas:**
 - Total de livros adicionados, lidos, lendo, para ler
 - Percentual de engajamento (lidos + lendo / total)
 - Estimativa de tempo de leitura (baseada em ~250 WPM)
- **Seção "Lendo Agora"** com cards dos livros em andamento, incluindo:
 - Capa, título, autor
 - Percentual concluído com barra de progresso
 - Link "Retomar Leitura" para `/leitor/{id}`

Integração com API: Real — `GET /user-books` (cálculos feitos no cliente).

10.1.5 Leitor de EPUB (`/leitor/[id]`)

Objetivo: Leitura de eBooks em formato EPUB diretamente no navegador com progresso sincronizado.

Componentes principais:

- **Header** com botão "Voltar", título do livro, autor e indicador de progresso
- **Barra de progresso** fina abaixo do header (clique alterna entre %, minutos restantes, páginas restantes)
- **Leitor EPUB** (biblioteca `react-reader`) com:
 - Carregamento via proxy `/api/epub` (evita CORS da Gutendex)
 - Cache em IndexedDB com política LRU
 - Sincronização automática de progresso a cada 2 segundos via `PUT /user-books/{id}`
 - Persistência de localização (`localStorage`) para retomar leitura
 - Heartbeat a cada 2 minutos para atualizar `lastAccessed` no cache

Integração com API: Real — GET /books/{id}, GET /user-books, PUT /user-books/{id}. **Cache:** IndexedDB com política LRU (30 itens, 250MB) + memória RAM.

10.1.6 Leitor Estático (/leitor)

Objetivo: Placeholder do leitor para visualização de design sem dependência de API.

Componentes principais:

- Header com controles de zoom e marcador
- Área de visualização com conteúdo mockado
- Navegação de página anterior/próxima

Fonte dos dados: Mock estático (data/books.ts).

10.1.7 Configurações (/configuracoes)

Objetivo: Gerenciamento de perfil e senha do usuário.

Componentes principais:

- **Seção Conta:** exibe email (somente leitura), campos editáveis de nome, sobrenome e username
- **Seção Senha:** campos para senha atual, nova senha e confirmação
- **Seções futuras:** Notificações e Aparência (placeholder "Em breve")

Integração com API: Real — GET /profile/me, PUT /profile/me, PUT /profile/password.

10.1.8 Suporte (/suporte)

Objetivo: FAQ e contato.

Componentes principais:

- Perguntas frequentes (como adicionar livro, acompanhar progresso, idiomas disponíveis)
- Contato por email (suporte@alexandria.pucsp.br)

- Sobre o projeto (descrição institucional)

Fonte dos dados: Conteúdo estático.

10.1.9 Login e Cadastro (/login, /registrar)

Objetivo: Autenticação de usuários.

Componentes principais:

- Formulário de login com username e senha
- Formulário de registro com username, nome, sobrenome, email e senha
- Modal de login (acessível de qualquer página não autenticada via `?auth=1` ou botão "Entrar")
- Botão "Entrar com Google" via `@react-oauth/google`

Integração com API: Real — `POST /auth/login`, `POST /auth/register`, `POST /auth/google`.

10.2 Telas Mobile

As telas mobile são exibidas em dispositivos com largura inferior a 768px. O layout substitui a sidebar por um **header compacto fixo** no topo e uma **barra de navegação inferior** com as seções principais (Explorar, Biblioteca, Dashboard).

As funcionalidades são as mesmas do desktop, com adaptações de layout:

- **Explorar** — hero simplificado, curadorias em grid vertical, lista em vez de grade
 - **Biblioteca** — grade em 2 colunas (vs 4), filtros em scroll horizontal
 - **Dashboard** — cards de estatísticas empilhados, "Lendo Agora" em lista vertical
 - **Leitor** — área de leitura em largura total, navegação por toque
-

11. Design e Interface

11.1 Origem do Design

O design do Alexandria foi elaborado no **Figma** e integrado ao desenvolvimento via **Figma MCP**, que permitiu a extração automática de tokens de cor, tipografia, estrutura de componentes e código de referência React + Tailwind.

11.2 Sistema de Design

Paleta de Cores:

Nome	Hex	Aplicação
Cream	#fcf9f0	Background principal, sidebar
Cream Dark	#f6f3ea	Inputs, cards secundários
Cream Active	#f2eee1	Item de navegação ativo
Cream Border	#e5e2da	Bordas, divisores
Cream Book	#ebe8df	Background de capas de livros
Brown	#300d00	Texto principal, botões primários
Brown Soft	#43474d	Texto secundário
Slate	#4c6078	Texto de apoio, ícones
Terra	#954925	Acento, estrelas, destaques
Blue Light	#d1e4ff	Badges

Tipografia:

Fonte	Peso	Aplicação
Manrope	300–800	Corpo de texto, labels, botões

Fonte	Peso	Aplicação
Playfair Display	400, 700	Títulos de seção, headings de livro
Noto Serif	700	Logotipo, título hero

Espaçamento e Grid:

- Sidebar: 256px (fixa)
- Conteúdo: largura máxima de 1280px
- Padding: 32px (desktop), 24px (mobile)
- Gap padrão de seção: 48px (desktop), 32px (mobile)

12. Autenticação e Segurança

12.1 Fluxo de Autenticação

1. POST /auth/register → Cria usuário (senha com BCrypt)
2. POST /auth/login → Valida credenciais, retorna token JWT
3. POST /auth/google → Valida token Google, retorna token JWT
4. Header em requests autenticadas:

Authorization: Bearer <token>

12.2 JWT

Propriedade	Descrição	Padrão
<code>jwt.secret</code>	Chave HMAC-SHA (min 256 bits)	Variável de ambiente
<code>jwt.expiration-ms</code>	Expiração do token	24h (86400000ms)
Claims	<code>userId</code> , <code>username</code> , <code>role</code>	—

12.3 Controle de Acesso por Papéis

Role	Acesso
Público	GET /books/**, /auth/**, /actuator/health, Swagger
USER	/user-books/**, /profile/**
ADMIN	Tudo que USER tem + POST/PUT/DELETE /books/**, /api/jobs/**

12.4 Google OAuth

O fluxo de login social segue estas etapas:

1. Frontend exibe botão "Entrar com Google" via `@react-oauth/google`
2. Usuário faz login na janela do Google e recebe um `credential` token
3. Frontend envia o token para `POST /auth/google`
4. Backend valida o token na API
`https://oauth2.googleapis.com/tokeninfo?id_token=...`
5. Verifica se `aud` (audience) corresponde ao `google.client-id` configurado
6. Busca usuário por email — se existir, retorna token JWT; se não, cria novo usuário automaticamente

12.5 Armazenamento no Cliente

- `localStorage`: `auth-token` (JWT) e `auth-user` (JSON com `userId` + `username`)
- `cookie`: `auth-token` (para middlewares Next.js, expira em 24h)

13. Integrações Externas

13.1 Gutendex API

- **Endpoint:** `https://gutendex.com/books`
- **Propósito:** Importar livros do Projeto Gutenberg (domínio público)
- **Formato:** JSON paginado (32 livros por página)
- **Parâmetros usados:** `?page={n}`, `?search={query}`
- **Mapeamento:** `GutendexMapper` converte `GutendexBookResponse` → `BookData` do domínio

13.2 Google OAuth 2.0

- **Endpoint:** https://oauth2.googleapis.com/tokeninfo?id_token={token}
- **Propósito:** Validar tokens de login social Google
- **Validação:** Verifica `aud` (clientId) e extrai `email`, `given_name`, `family_name`

13.3 AWS EventBridge Scheduler

- **Propósito:** Disparar o job de sincronização Gutendex em intervalos regulares
 - **Ação:** `POST /api/jobs/sync-gutendex` (requer `ROLE_ADMIN`)
-

14. Infraestrutura e Deploy

14.1 Desenvolvimento Local (Docker Compose)

Serviços:

- alexandria-db (MySQL 8, porta 3306)
- alexandria-api (Spring Boot, porta 8080)

14.2 Produção (AWS)

Serviço	Função
ECS Fargate	Container da API Spring Boot
RDS MySQL	Banco de dados gerenciado
EventBridge Scheduler	Job recorrente de sincronização
Parameter Store	Configurações (DB_PASSWORD, JWT_SECRET, etc.)
ECR	Registry de imagens Docker

14.3 Kubernetes

O projeto inclui manifestos K8s para orquestração alternativa (deploy, service, configmap, secrets).

14.4 Variáveis de Ambiente

Variável	Obrigatória	Padrão	Descrição
DB_HOST	Não	localhost	Host do MySQL
DB_PORT	Não	3306	Porta do MySQL
DB_NAME	Não	alexandriadb	Nome do database
DB_USER	Sim (RDS)	root	Usuário do MySQL
DB_PASSWORD	Sim (RDS)	root	Senha do MySQL
JWT_SECRET	Sim (RDS)	dev-secret-...	Chave JWT (min 256 bits)
JWT_EXPIRATION_MS	Não	86400000	Expiração do token
CORS_ALLOWED_ORIGINS	Não	http://localhost:3000	Origens CORS
GOOGLE_CLIENT_ID	Sim	—	Google OAuth Client ID

15. Estrutura do Projeto

alexandria/

├── README.md

├── docs/

| └── documentacao.md # Este documento

|

├── alexandria-frontend/ # Aplicação Next.js

| └── public/

- | | |— manifest.json # PWA manifest
- | | |— sw.js # Service Worker
- | | |— icon-192x192.png
- | | |— icon-512x512.png
- | |— src/
- | | |— app/
- | | | |— globals.css # Tokens Tailwind @theme
- | | | |— layout.tsx # Root layout + providers
- | | | |— page.tsx # Redirect → /explorar
- | | | |— api/epub/route.ts # Proxy de download EPUB
- | | | |— (auth)/ # Grupo: login, registrar
- | | | | |— layout.tsx
- | | | | |— login/page.tsx
- | | | | |— registrar/page.tsx
- | | | |— (main)/ # Grupo: rotas com sidebar
- | | | |— layout.tsx # Sidebar + header + bottom nav
- | | | |— explorar/
- | | | | |— page.tsx
- | | | | |— [id]/page.tsx
- | | | |— biblioteca/page.tsx
- | | | |— dashboard/page.tsx
- | | | |— leitor/

- | | | | | — page.tsx
- | | | | | — [id]/page.tsx
- | | | | — configuracoes/page.tsx
- | | | | — suporte/page.tsx
- | | | — components/ # Componentes reutilizáveis
- | | | | — Sidebar.tsx
- | | | | — Header.tsx
- | | | | — MobileHeader.tsx
- | | | | — BottomNav.tsx
- | | | | — BookCard.tsx
- | | | | — LoginModal.tsx
- | | | | — AuthModalTrigger.tsx
- | | | | — GoogleProvider.tsx
- | | | | — ServiceWorkerRegister.tsx
- | | | — contexts/
- | | | | — AuthContext.tsx
- | | | | — AuthModalContext.tsx
- | | | — hooks/
- | | | | — useEpub.ts
- | | | — lib/
- | | | | — api.ts # Cliente HTTP completo

```

| | | └─ epub-cache.ts      # Cache IndexedDB + LRU
| | | └─ proxy.ts
| | └─ data/
| | └─ books.ts            # Dados mockados
| └─ package.json
| └─ tsconfig.json
| └─ next.config.ts
| └─ tailwind.config.ts
|
└─ alexandria-backend/      # API Spring Boot
| └─ src/main/java/com/pucsp/alexandria/
| | └─ AlexandriaApplication.java
| | └─ domain/
| | | └─ book/              # Book, BookId, BookSource, BookRepository, etc.
| | | └─ author/           # Author, AuthorId, AuthorRepository
| | | └─ user/              # User (com Role), UserId, UserRepository
| | | └─ userbook/          # UserBooks, UserBooksStatus, UserBooksRepository
| | | └─ shared/            # Id, Email, AuthenticatedUser, Port, DomainException
| | └─ application/
| | | └─ book/              # 7 use cases
| | | └─ auth/              # 3 use cases (Register, Authenticate, Google)
| | | └─ profile/           # 3 use cases (Get, Update, Password)

```

- | | | └─ userbooks/ # 5 use cases
- | | └─ adapter/
- | | | └─ in/rest/ # 5 controllers + DTOs
- | | | └─ out/persistence/ # Impls, JPA, Mappers, GutendexClient
- | | └─ config/ # Security, JWT, CORS, Async, OpenAPI
- | | └─ advice/ # GlobalExceptionHandler
- | └─ src/main/resources/
- | | └─ application.properties
- | | └─ application-rds.properties
- | | └─ db/migration/ # V0, V001, V002
- | └─ markdown/ # Documentação técnica
- | | └─ C4_MODEL.md
- | | └─ ARCHITECTURE.md
- | └─ Dockerfile
- | └─ docker-compose.yml
- | └─ pom.xml
- | └─ k8s/ # Manifestos Kubernetes

16. Como Executar o Projeto

16.1 Pré-requisitos

- **Node.js** 18+ e **npm**

- **Java 17+ e Maven**
- **Docker e Docker Compose** (para banco MySQL)
- Conexão com a internet

16.2 Backend

1. Iniciar MySQL com Docker

```
cd alexandria-backend
```

```
docker-compose up -d
```

2. Executar a aplicação

```
./mvnw spring-boot:run
```

API disponível em: <http://localhost:8080>

Swagger UI: <http://localhost:8080/swagger-ui.html>

16.3 Frontend

1. Instalar dependências

```
cd alexandria-frontend
```

```
npm install
```

2. Executar em desenvolvimento

```
npm run dev
```

Aplicação disponível em: <http://localhost:3000>

16.4 Build de Produção

Backend

```
cd alexandria-backend
```

```
./mvnw package -DskipTests
```

```
docker build -t alexandria-api .
```

Frontend

cd alexandria-frontend

npm run build

npm start

16.5 Docker Completo

cd alexandria-backend

docker-compose up -d # Sobe MySQL + API

17. Testes

17.1 Backend

O backend possui testes unitários e de integração divididos por camada:

src/test/java/com/pucsp/alexandria/

- |— domain/ # Testes de entidades e Value Objects (JUnit)
- |— application/ # Testes de Use Cases (Mockito)
- |— adapter/
- | |— out/persistence/ # Testes de repositórios (H2 + Testcontainers)
- | |— in/rest/ # Testes de integração dos controllers
- |— advice/ # Testes do GlobalExceptionHandler
- |— config/jwt/ # Testes de JWT

Ferramentas: JUnit 5, Mockito, H2 (banco em memória), Testcontainers (MySQL real), JaCoCo (cobertura).

Executar todos os testes

```
cd alexandria-backend
```

```
./mvnw test
```

```
# Gerar relatório de cobertura
```

```
./mvnw verify
```

```
# Relatório em: target/site/jacoco/index.html
```

17.2 Frontend

Testes podem ser executados com ferramentas como Jest ou Playwright (ainda não configurados no projeto).

18. Plano de Evolução

Curto Prazo

Item	Status
Ativar Flyway no perfil RDS para gerenciamento versionado	 Pendente
Adicionar suporte a temas (dark mode)	 Pendente
Testes automatizados no frontend (Jest + Playwright)	 Pendente

Médio Prazo

Item	Status
GitHub Actions para CI/CD	 Pendente
Endpoint de notificações (push notifications)	 Pendente
Upload de capas personalizadas	 Pendente
Sugestões personalizadas de leitura baseadas no histórico	 Pendente

Item	Status
Exportação do acervo em CSV/PDF	 Pendente

Longo Prazo

Item	Status
Clube de leitura com funcionalidades sociais	 Pendente
Recomendações colaborativas (entre usuários)	 Pendente
Integração com outras fontes de livros (Open Library, Google Books)	 Pendente
Aplicativo mobile nativo (React Native)	 Pendente

19. Considerações Finais

O projeto Alexandria representa uma aplicação web full-stack moderna que integra boas práticas de desenvolvimento com um design cuidadosamente elaborado.

Pontos Fortes

- **Arquitetura limpa** no backend — Clean Architecture com Ports & Adapters garante baixo acoplamento e alta testabilidade
- **Design fiel ao protótipo** Figma, com tokens de cor e tipografia consistentes
- **Experiência completa** — do cadastro/login à leitura de eBooks com progresso sincronizado
- **PWA** — instalação no dispositivo, cache offline de EPUBs
- **Autenticação social** — Google OAuth integrado para onboarding simplificado
- **Performance otimizada** — Next.js (fontes, imagens, bundle splitting), cache IndexedDB, compressão no backend
- **Cobertura de testes** no backend com JUnit, Mockito, Testcontainers e JaCoCo
- **Infraestrutura cloud-ready** — Docker, AWS ECS, RDS, EventBridge, K8s
- **11 páginas funcionais**, sendo a maioria com dados reais da API

Diferenciais

- Leitor de EPUB real com **react-reader**, cache LRU offline e sincronização automática de progresso

- Controle de acesso por papéis (ADMIN/USER) com endpoints protegidos
 - Sincronização programada do acervo da Gutendex via job assíncrono
 - Dashboard com estatísticas reais de leitura (tempo estimado, engajamento)
 - Modal de login universal com parâmetro `?auth=1` acionável de qualquer página
-

Documentação gerada em junho de 2026.